```
    std::shared_ptr<T> pop()
    {
        std::lock_guard<std::mutex> lock(m);
        if(data.empty()) throw empty_stack();    <-- 2
        std::shared_ptr<T> const res(
           std::make_shared<T>(std::move(data.top())));    <-- 3
        data.pop();          <-- 4
        return res;
    }
    void pop(T& value)
    {
        std::lock_guard<std::mutex> lock(m);
        if(data.empty()) throw empty_stack();
        value=std::move(data.top());          <-- 5
        data.pop();                  <-- 6
    }
    bool empty() const
    {
        std::lock_guard<std::mutex> lock(m);
        return data.empty();
    }
};
```

Let's look at each of the guidelines in turn, and see how they apply here.

First, as you can see, the basic thread safety is provided by protecting each member function with a lock on the mutex, `m`. This ensures that only one thread is actually accessing the data at any one time, so provided each member function maintains the invariants, no thread can see a broken invariant.

Second, there's a potential for a race condition between `empty()` and either of the `pop()` functions, but because the code explicitly checks for the contained stack being empty while holding the lock in `pop()`, this race condition isn't problematic. By returning the popped data item directly as part of the call to `pop()`, you avoid a potential race condition that would be present with separate `top()` and `pop()` member functions such as those in `std::stack<>`.

Next, there are a few potential sources of exceptions. Locking a mutex may throw an exception, but not only is this likely to be exceedingly rare